Metaheurísticas

Seminario 2. Problemas de optimización con técnicas basadas en búsqueda local

Problema del Portolio

1. Agrupamiento Clásico
2. Problema del Portfolio: Definición y Variantes del Problema
3. Análisis del Problema
4. Evaluación de Soluciones
5. Generación y Reparación de Soluciones
6. Solución Greedy
7. Búsquedas por Trayectorias Simples
8. Casos del problema

El Problema del Agrupamiento

- El **agrupamiento** o **análisis de clusters** o ***clustering*** en inglés, persigue la clasificación de objetos de acuerdo a posibles similitudes entre ellos.

- Es una técnica de aprendizaje no supervisado que permite descubrir grupos (inicialmente desconocidos) en un conjunto de datos o agrupar objetos similares entre sí

- Su principal aplicación es la clasificación en entornos en los que las clases son desconocidas a priori

- Existen otras muchas aplicaciones, tales como la clasificación en entornos con clases cambiantes o la reducción de la complejidad de una muestra inicial de ejemplos

El Problema del Agrupamiento

- Son ejemplos típicos de análisis de clusters en Informática e Ingeniería los caracteres escritos a mano, las muestras de diálogo, las huellas dactilares y las imágenes

- En Ciencias de la Vida los objetos de análisis son plantas, animales e insectos. El agrupamiento se puede aplicar desde a la creación de taxonomías completas hasta la clasificación de especies en subespecies

- En Biología, Química y Farmacología está muy extendido el uso de agrupamiento para moléculas, proteínas, genes, etc.

- También se aplica en Ciencias de la Información, Empresariales y Políticas: documentos, votos, análisis de mercados, segmentación de clientes, programas de ventas, etc.

¿Qué es un Cluster?

- Cada grupo de objetos similares entre sí obtenidos tras el proceso se denomina **cluster**. Al conjunto de clusters resultante se le denomina **configuración de clusters**

- El termino **similitud** se suele entender en su sentido matemático, es decir, definido mediante una **distancia**, habitualmente la Euclídea

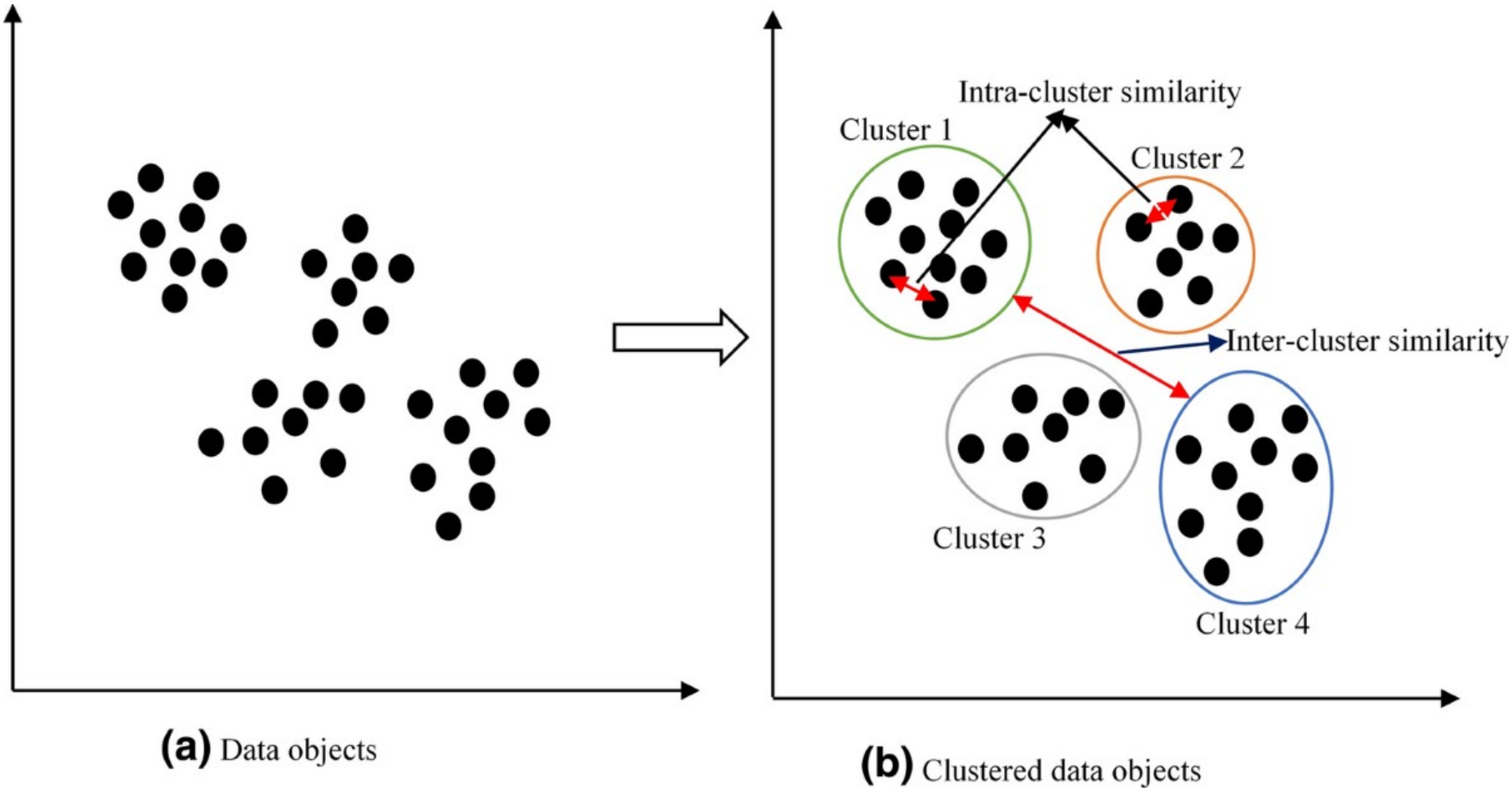
- De este modo, cada cluster se caracteriza por un **centroide**, centro geométrico del cluster en el espacio d-dimensional de características de los objetos

- Cada objeto pertenece a un único cluster y su distancia Euclídea al centroide de éste es siempre menor que a los restantes

Criterio para valorar *clustering*

- Distancia intra-clustering:
  - Distancia entre los elementos del mismo cluster.
- Distancia inter-clustering:
  - Distancia entre los distintos clusters.
- Queremos minimizar la distancia intra-cluster y/o maximizar la distancia extra-cluster.

Ejemplo de Agrupamiento

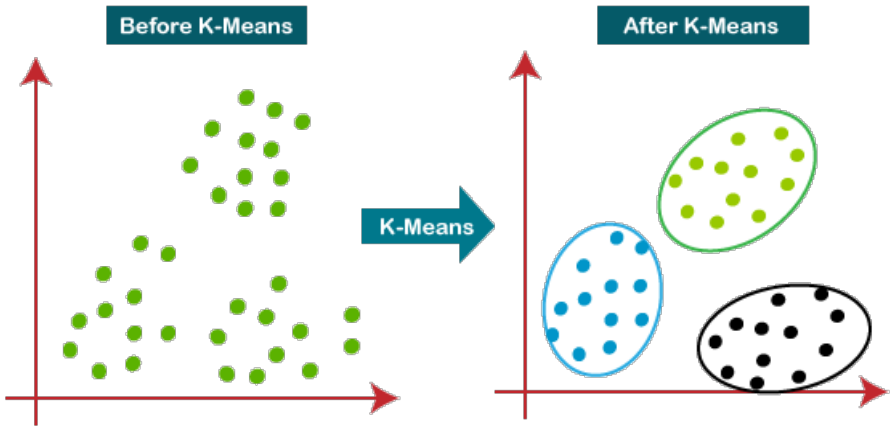


Agrupamiento con Restricciones: Definición del Problema

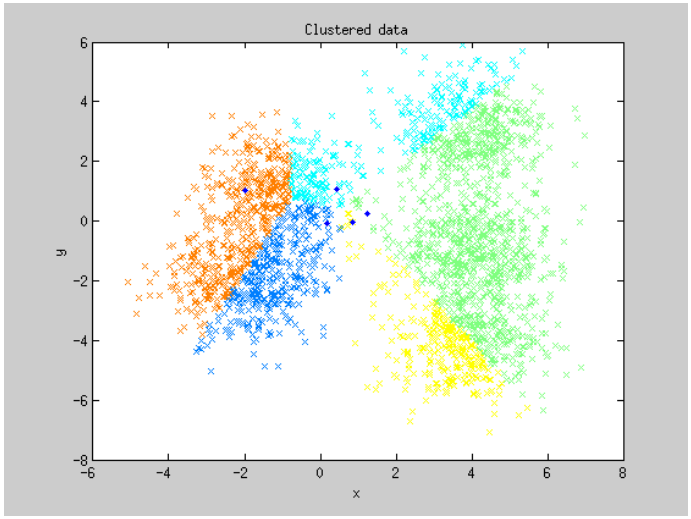
- Se puede formular en términos de optimización combinatoria. Dicha formulación es sencilla pero de compleja resolución (**es NP-completo**)

- Entre algunos ejemplos de aplicación del PAR encontramos:
  - robótica aplicada,
  - márketing aplicado,
  - detección de comunidades terroristas,
  - diseño de distritos electorales,
  - análisis de la apnea obstructiva del sueño, y
  - detección de carriles de vehículos en aplicaciones GPS, entre otras

Algoritmos para Clustering



- K-Means: C clusters.
  1) crea C puntos.
  2) Asigna los más cercanos.
  3) Actualiza centroides.



Clustering con Restricciones

- *Restricciones de instancia*: Dada una pareja de instancias, se establece una restricción de tipo **Must-Link (ML), si deben pertenecer al mismo cluster**, o de tipo **Cannot-Link (CL), si no pueden pertenecer al mismo cluster.**
- K-Means y otros no pueden abordarlo.
- Recientemente se ha abordado método exacto [1], pero el tiempo es excesivo (12 minutos con tamaño 250).
- Se hace necesario el uso de MHs para abordarlo con confianza.

V. Piccialli, A. Russo Russo, y A. M. Sudoso, «An exact algorithm for semi-supervised minimum sum-of-squares clustering», Computers & Operations Research, vol. 147, p. 105958, nov. 2022, doi: 10.1016/j.cor.2022.105958.

Variantes del Problema

Existen variantes del problema que dependen del **modo en que se interpretan las restricciones**:

- *Restricciones Fuertes (Hard)*: Todas las restricciones deben satisfacerse en la partición *C* del conjunto de datos *X*. Se dice que una partición es **factible si y solo si cumple con todas las restricciones en *R***
- *Restricciones Débiles (Soft)*: La partición C del conjunto de datos X debe minimizar el número de restricciones incumplidas pero puede incumplir algunas
- Nosotros abordaremos el problema como restricción débil, ya que hay muchas restricciones.
- Queremos minimizar también el número de restricciones violadas.

Representación del problema

- Tenemos un conjunto X de vectores, y queremos asignarle a cada elemento/vector un cluster entre 0 y C-1.
- Buscaremos reducir la distancia intra-cluster, y reducir el número de restricciones violadas.
- ¿Cómo se calcula la distancia intra-cluster?
  - Obtenemos el promedio de los elementos del mismo cluster.
  - Para cada cluster calculamos la distancia a su centro.
  - Calculamos el promedio de dichas distancias.
- Para el cálculo de violaciones:
  - Contamos el número de restricciones incumplidas.
- Valoración final: Media ponderada entre ambos criterios.

Análisis del Problema

- El problema del agrupamiento simple es NP-Duro. Al añadirle restricciones de tipo ML y CL pasa a ser NP-Completo

- Resolverlo de forma óptima es intratable, debemos buscar soluciones aproximadas

- La representación de las restricciones en forma de lista es más eficiente cuando es necesario recorrer las restricciones para realizar operaciones sobre ellas, ya que solo es necesario recorrer una lista de longitud $|R|$ en lugar de recorrer una matriz de n^2 posiciones

- La representación en forma de matriz es más eficiente cuando se sabe que restricción se quiere comprobar, ya que las matrices implementan acceso aleatorio

Análisis del Problema

- **COMENTARIOS:**

✓ *k* debe ser fijo y conocido a priori. El aprendizaje del número óptimo de clusters para un conjunto de datos concreto es un problema complejo

✓ La función $\text{minimizar}\{\bar{C}\}$ puede presentar muchos mínimos locales

- Para diseñar un método de resolución del PAR es necesario disponer de dos componentes:

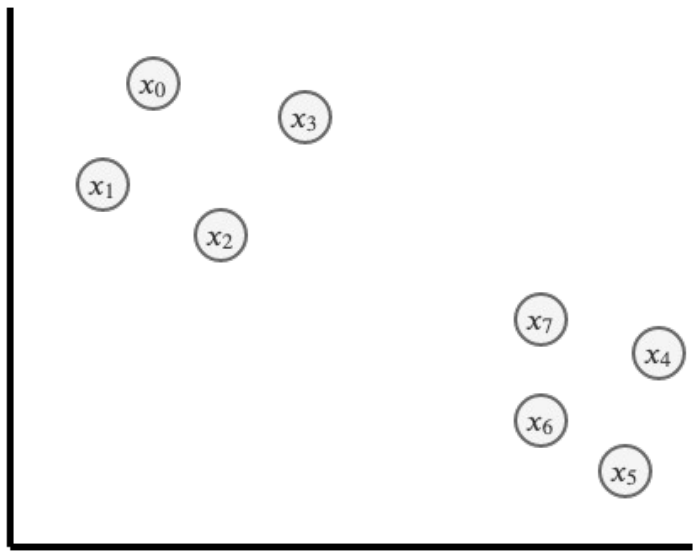
a) Una medida de la bondad con la que la configuración de clusters actual representa la muestra original de instancias (habitualmente el valor $\bar{C}$ combinado con $infeasibility$)

b) Un algoritmo que busque la configuración de clusters que optimice dichas medidas

Definición del Problema: Ejemplo

Consideramos el conjunto de datos X formado por 8 instancias en dos dimensiones ($n = 8$, $d = 2$).

$$X = \begin{pmatrix} x_{0,1} & x_{0,2} \\ x_{1,1} & x_{1,2} \\ x_{2,1} & x_{2,2} \\ x_{3,1} & x_{3,2} \\ x_{4,1} & x_{4,2} \\ x_{5,1} & x_{5,2} \\ x_{6,1} & x_{6,2} \\ x_{7,1} & x_{7,2} \end{pmatrix} \Rightarrow$$

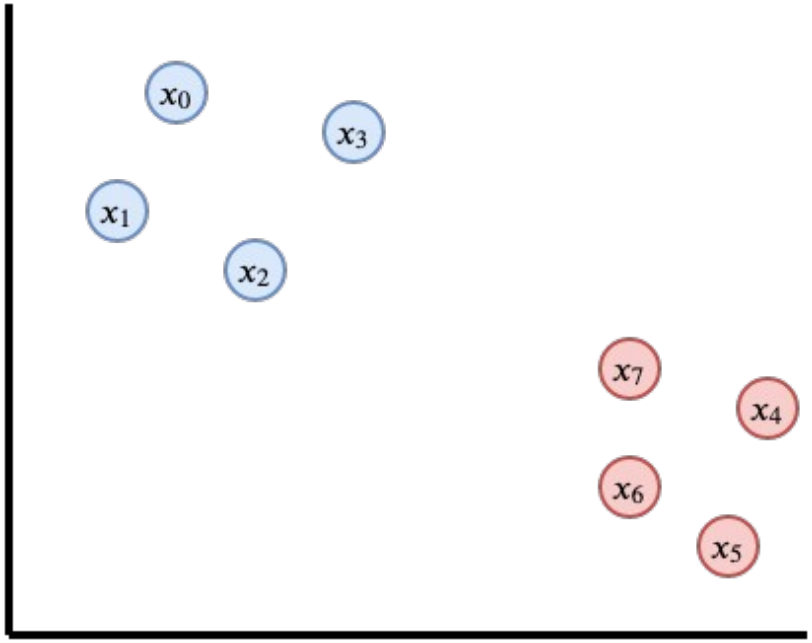


Definición del Problema: Ejemplo

Consideramos la partición C que produce los clusters c_1 y c_2 asignando etiquetas a las instancias de X (**azul** y **rojo**):

$$c_1 = \{\overrightarrow{x_0}, \overrightarrow{x_1}, \overrightarrow{x_2}, \overrightarrow{x_3}\}$$

$$c_2 = \{\overrightarrow{x_4}, \overrightarrow{x_5}, \overrightarrow{x_6}, \overrightarrow{x_7}\}$$

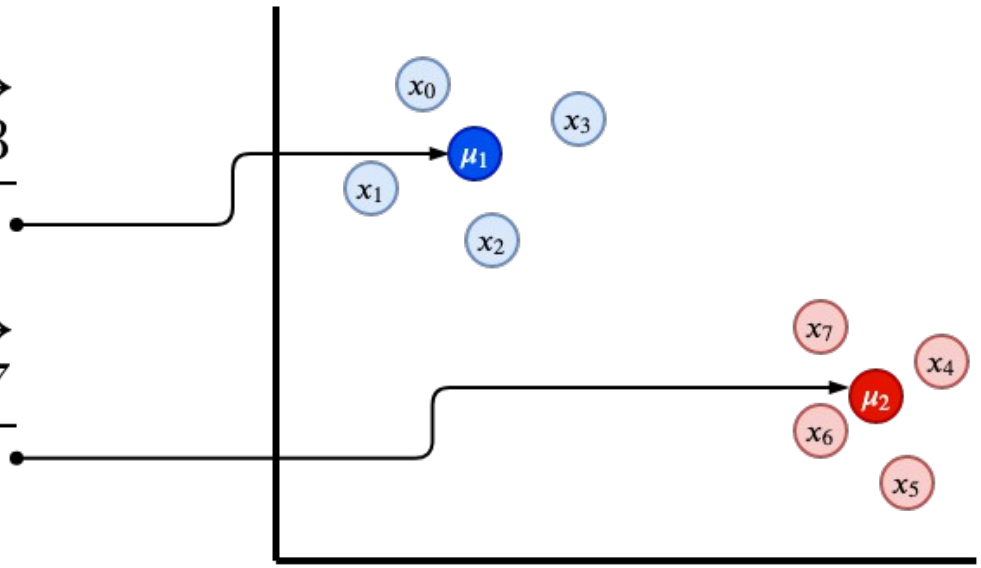


Definición del Problema: Ejemplo

De esta forma los centroides $\overrightarrow{\mu_1}$ y $\overrightarrow{\mu_2}$ se calculan como:

$$\overrightarrow{\mu_1} = \frac{\overrightarrow{x_0} + \overrightarrow{x_1} + \overrightarrow{x_2} + \overrightarrow{x_3}}{4}$$

$$\overrightarrow{\mu_2} = \frac{\overrightarrow{x_4} + \overrightarrow{x_5} + \overrightarrow{x_6} + \overrightarrow{x_7}}{4}$$



Definición del Problema: Ejemplo

Las distancias medias intra-cluster $\overline{c_1}$ y $\overline{c_2}$ se calculan como:

$$\overline{c_1} = \frac{\|\overrightarrow{x_0} - \overrightarrow{\mu_1}\|_2 + \|\overrightarrow{x_1} - \overrightarrow{\mu_1}\|_2 + \|\overrightarrow{x_2} - \overrightarrow{\mu_1}\|_2 + \|\overrightarrow{x_3} - \overrightarrow{\mu_1}\|_2}{4}$$

$$\overline{c_2} = \frac{\|\overrightarrow{x_4} - \overrightarrow{\mu_2}\|_2 + \|\overrightarrow{x_5} - \overrightarrow{\mu_2}\|_2 + \|\overrightarrow{x_6} - \overrightarrow{\mu_2}\|_2 + \|\overrightarrow{x_7} - \overrightarrow{\mu_2}\|_2}{4}$$

Y la desviación general $\bar{C}$ se calcula como:

$$\overline{C} = \frac{\overline{c_1} + \overline{c_2}}{2}$$

Definición del Problema: Formalización

- Durante el desarrollo de la práctica trabajaremos con las restricciones a nivel de instancias Must-Link (ML) y Cannot-Link (CL):

$$R = ML \cup CL$$

- Podemos formalizar las restricciones como sigue:
  - Restricciones Must-Link $ML(\vec{x_i}, \vec{x_j})$: las instancias $\vec{x_i}$ y $\vec{x_j}$ deben ser asignadas al mismo cluster
  - Restricciones Cannot-Link $CL(\vec{x_i}, \vec{x_j})$: las instancias $\vec{x_i}$ y $\vec{x_j}$ no pueden ser asignadas al mismo cluster

Definición del Problema: Formalización

- Notamos el número de restricciones ML con $|ML|$ y el de restricciones CL con $|CL|$, de forma que el número total de restricciones $|R| = |ML| + |CL|$

- Notamos la i-ésima restricción ML como ML_i. Notamos la i-ésima restricción CL como CL_i

- Notamos la primera instancia implicada en ML_i como $\overrightarrow{ML_{[i,1]}}$ y la segunda como $\overrightarrow{ML_{[i,2]}}$. Notamos la primera instancia implicada en CL_i como $\overrightarrow{CL_{[i,1]}}$ y la segunda como $\overrightarrow{CL_{[i,2]}}$.

- Definimos $h_C(\cdot)$ como la función que dada una instancia $\overrightarrow{x_i}$ devuelve la etiqueta l_j asociada al cluster c_j al que $\overrightarrow{x_i}$ pertenece según la partición C

Definición del Problema: Formalización

- Dada una partición C y los conjuntos de restricciones ML y CL, definimos $infeasibility$ ("grado de infactibilidad") como el número de restricciones que C incumple:

$$infeasibility = \sum_{i=0}^{|ML|} \mathbb{1}(h_C(\overrightarrow{ML_{[i,1]}}) \neq h_C(\overrightarrow{ML_{[i,2]}})) + \sum_{i=0}^{|CL|} \mathbb{1}(h_C(\overrightarrow{CL_{[i,1]}}) = h_C(\overrightarrow{CL_{[i,2]}}))$$

donde $\mathbb{1}$ es la función indicadora, que devuelve 1 en caso de ser cierta la expresión Booleana que toma como argumento y 0 en otro caso

Definición del Problema: Formalización

- Dado que disponemos de estructuras para almacenar las restricciones, podemos calcular el valor para $infeasibility$ de manera sencilla y eficiente

- Definimos $V(\overrightarrow{x_i}, \overrightarrow{x_j})$ como la función que, dada una pareja de instancias, devuelve 1 si la asignación de dichas instancias viola una restricción y 0 en otro caso

- Podemos calcular $infeasibility$ como el número de restricciones incumplidas por una partición C del conjunto de datos X dada una matriz de restricciones MR o una lista de restricciones LR:

$$infeasibility = \sum_{i=0}^{n} \sum_{j=i+1}^{n} V(\overrightarrow{x_i}, \overrightarrow{x_j})$$

Función Objetivo de una solución

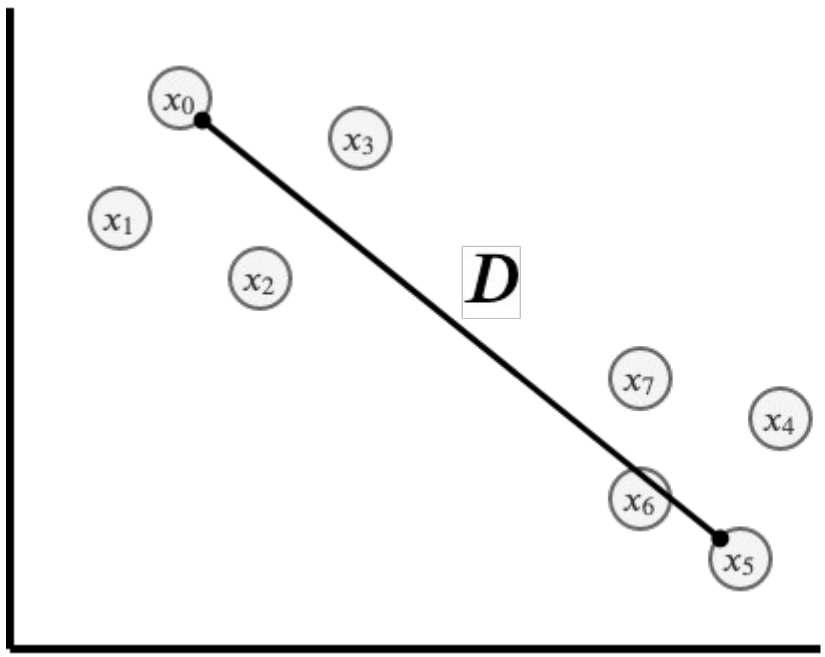
Al utilizar restricciones débiles tenemos que plantear una función objetivo que optimice a la vez la desviación general y el número de restricciones incumplidas (minimización):

$$f = \overline{C} + (infeasibility * \lambda)$$

La función propuesta utiliza el parámetro λ como parámetro de escalado para dar relevancia al término $infeasibility$. Si se establece correctamente, la búsqueda local optimiza primero el número de restricciones incumplidas y luego la desviación general

Función Objetivo

Para asegurar que el factor $infeasibility$ tiene la suficiente relevancia establecemos λ siempre como **el cociente entre un valor mayor a la distancia máxima existente en el conjunto de datos y el número de restricciones del problema**:



$$\lambda = \frac{\lceil D \rceil}{|R|}$$

Definición del Problema: Formalización

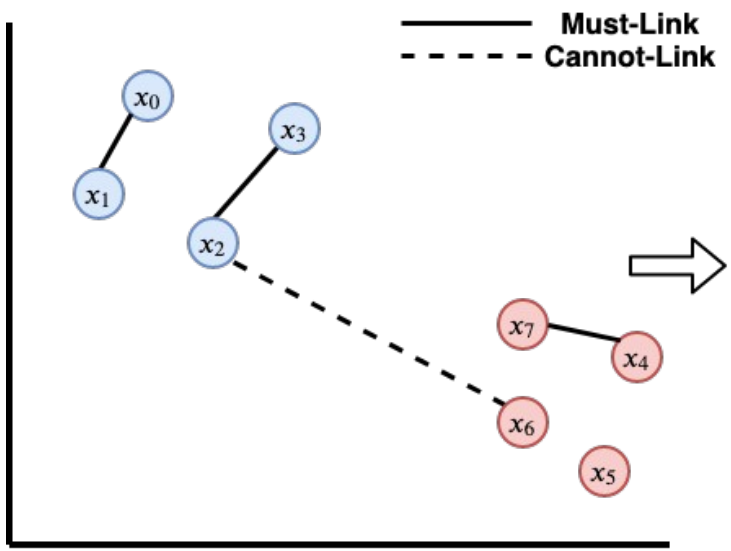
- Las restricciones pueden venir dadas en forma de lista de $|R|$ entradas (LR) o de matriz de n^2 entradas (MR)

- En formato lista las restricciones consiste en tripletas en las que las dos primeras posiciones contienen los identificadores de las instancias implicadas y el tercero contiene el tipo de restricción

- En formato matriz las restricciones consisten en valores dentro de la matriz que indican el tipo de restricción que existe entre las instancias asociadas a los índices que identifican el valor

Definición del Problema: Ejemplo

Las restricciones vienen dadas en forma de una matriz MR de n^2 entradas. Utilizamos un 1 para indicar restricción de tipo ML, un -1 para indicar CL y 0 para indicar que no hay restricción. La matriz es siempre **simétrica** y su diagonal contiene 1s:

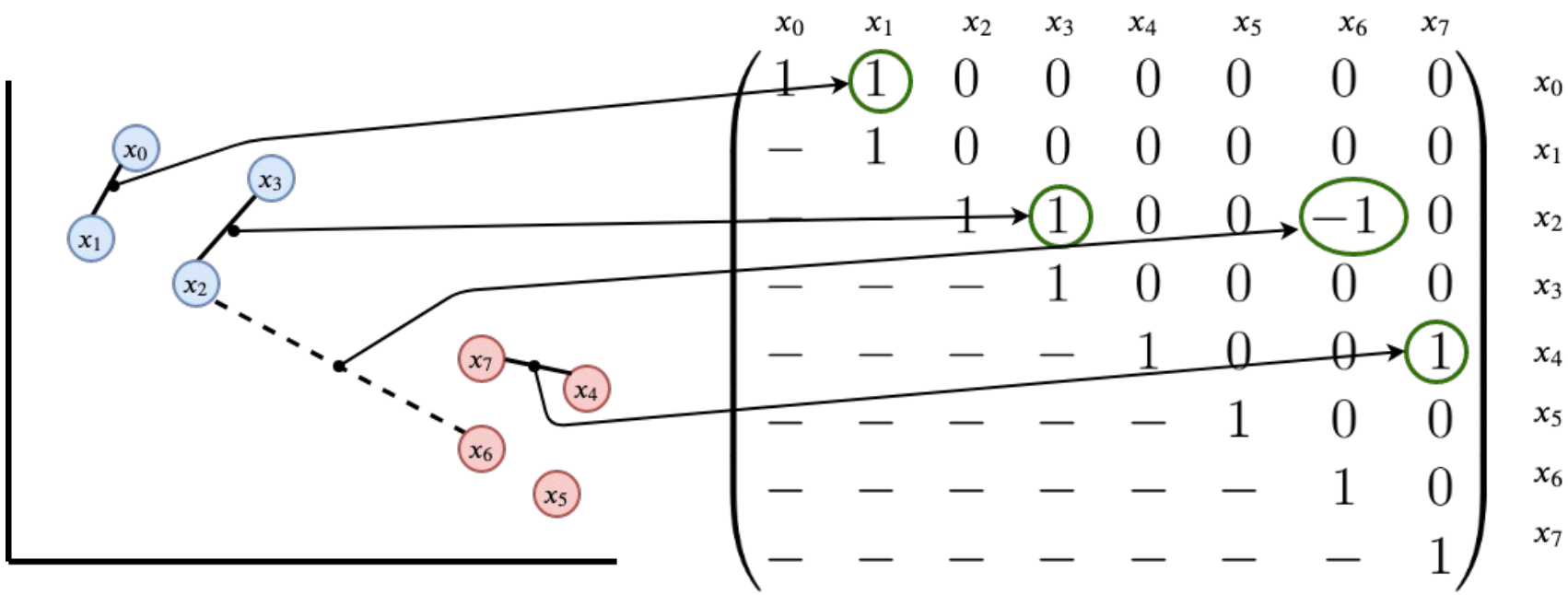


$$
\begin{array}{c c}
\begin{array}{cccccccc} x_0 & x_1 & x_2 & x_3 & x_4 & x_5 & x_6 & x_7 \end{array} & \\
\begin{pmatrix}
1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\
- & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\
- & - & 1 & 1 & 0 & 0 & -1 & 0 \\
- & - & - & 1 & 0 & 0 & 0 & 0 \\
- & - & - & - & 1 & 0 & 0 & 1 \\
- & - & - & - & - & 1 & 0 & 0 \\
- & - & - & - & - & - & 1 & 0 \\
- & - & - & - & - & - & - & 1
\end{pmatrix} &
\begin{array}{c} x_0 \\ x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \end{array}
\end{array}
$$

¡OJO! Hay que trabajar con la matriz completa de dimensión n^2

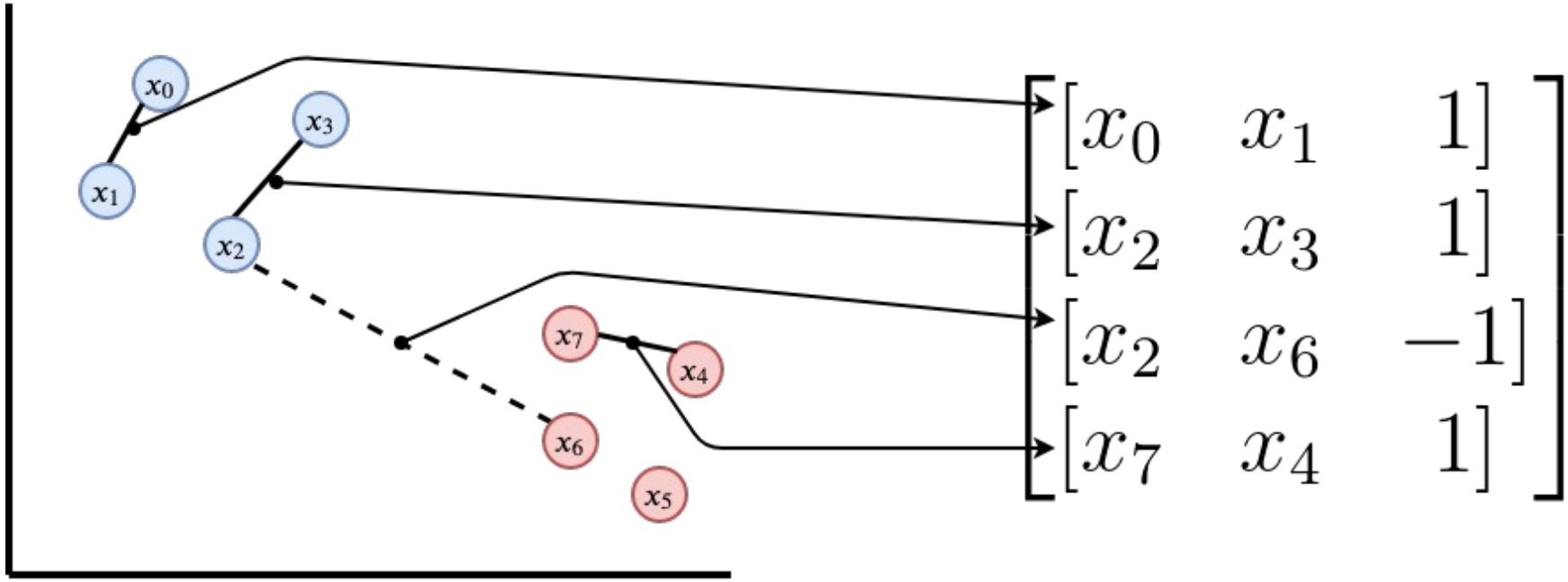
Definición del Problema: Matriz de restricciones

Las restricciones vienen dadas en forma de una matriz MR de n^2 entradas. Utilizamos un 1 para indicar restricción de tipo *ML*, un -1 para indicar *CL* y 0 para indicar que no hay restricción. La matriz es siempre simétrica y su diagonal contiene 1s.



Definición del Problema: Lista de restricciones

En formato lista, LR, las restricciones consiste en tripletas en las que las dos primeras posiciones contienen los identificadores de las instancias implicadas y el tercero contiene el tipo de restricción. Utilizamos un 1 para indicar una restricción *ML* y un -1 para indicar una *CL*:



Definición del Problema: Formalización

- Definimos la **desviación general de la partición** $C = \{c_1, \dots, c_k\}$ como la media de las desviaciones intra-cluster $\bar{C}$:

$$\overline{C} = \frac{1}{k} \sum_{c_i \in C} \overline{c_i}$$

- Para que una partición $C = \{c_1, \dots, c_k\}$ del conjunto de datos X sea válida debe cumplir que (**restricciones fuertes**):
  - Todos los clusters deben contener al menos una instancia:

$$c_i \neq \emptyset \forall i \in \{1, \cdots, k\} \leftrightarrow |c_i| > 0 \forall i \in \{1, \cdots, k\}$$

  - Cada instancia debe pertenecer a un solo cluster:

$$c_i \cap c_j = \emptyset \forall i, j \in \{1, \cdots, k\} | i \neq j$$

  - La unión de los clusters debe ser el conjunto de datos X:

$$\bigcup_{c_i \in C} c_i = X$$

¿Qué representación es mejor?

Definición del Problema: Formalización

- Podemos representar para cada elemento un valor numérico entre 0 y C-1 (o entre 1 y C).
- Esta representación asegura las restricciones fuertes.

$$C = \{\overset{x_0}{1}, \overset{x_1}{1}, \overset{x_2}{1}, \overset{x_3}{1}, \overset{x_4}{2}, \overset{x_5}{2}, \overset{x_6}{2}, \overset{x_7}{2}\}$$

$$S = [\overset{s_0}{1}, \overset{s_1}{1}, \overset{s_2}{1}, \overset{s_3}{1}, \overset{s_4}{2}, \overset{s_5}{2}, \overset{s_6}{2}, \overset{s_7}{2}]$$

$$C = \{\underline{\overset{x_0}{1}, \overset{x_1}{1}, \overset{x_2}{1}, \overset{x_3}{1}}, \underline{\overset{x_4}{2}, \overset{x_5}{2}, \overset{x_6}{2}, \overset{x_7}{2}}\}$$

$$c_1 = \{\overrightarrow{x_0}, \overrightarrow{x_1}, \overrightarrow{x_2}, \overrightarrow{x_3}\} \qquad c_2 = \{\overrightarrow{x_4}, \overrightarrow{x_5}, \overrightarrow{x_6}, \overrightarrow{x_7}\}$$

La Solución Greedy

La solución Greedy para el PAR consiste en **modificar el algoritmo k-medias para tener en cuenta dichas restricciones**. Para la solución greedy haremos una interpretación débil de las restricciones. De esta forma, **para cada instancia se llevará a cabo la asignación que menos restricciones viole y que más disminuya la desviación general** $\overline{C}$

La Solución Greedy: El Algoritmo

Algorithm 1: K-medias Restringido Débil

Input: Conjunto de datos X, conjunto de restricciones R, número de clusters k, centroides iniciales $\{\mu_1, \cdots, \mu_k\}$.

```
/* Barajamos los indices para recorrer X de forma aleatoria
   sin repeticion                                          */
```

[1] $RSI \leftarrow$ `RandomShuffle`$(\{1, \cdots, n\})$

[2] **do**

[3] **for** $i \in RSI$ **do**

[4]   - Calcular el incremento en $infeasibility$ que produce la asignación de x_i a cada cluster c_j.

[5]   - De entre las asignaciones que producen menos incremento en $infeasibility$, seleccionar la asociada con el centroide μ_j más cercano a x_i.

[6] **end**

[7] **for** $i \in \{1, \cdots, k\}$ **do**

[8]   Actualizar el centroide μ_i promediando las instancias de su cluster asociado c_i.

[9] **end**

[10] **while** *Se produzca cambio en* C;

[11] **return** $C = \{c_1, \cdots, c_k\}$

Algoritmo Greedy

- El algoritmo Greedy de Glover hace uso del concepto de elemento más central de un conjunto *X*:

$$s_central(X) = \frac{\sum_{s_i \in X} s_i}{|X|}$$

- El primer elemento seleccionado es el de mayor distancia del elemento más central

- Cada vez que se añade un nuevo elemento al conjunto de seleccionados *Sel*, se actualiza el elemento más central

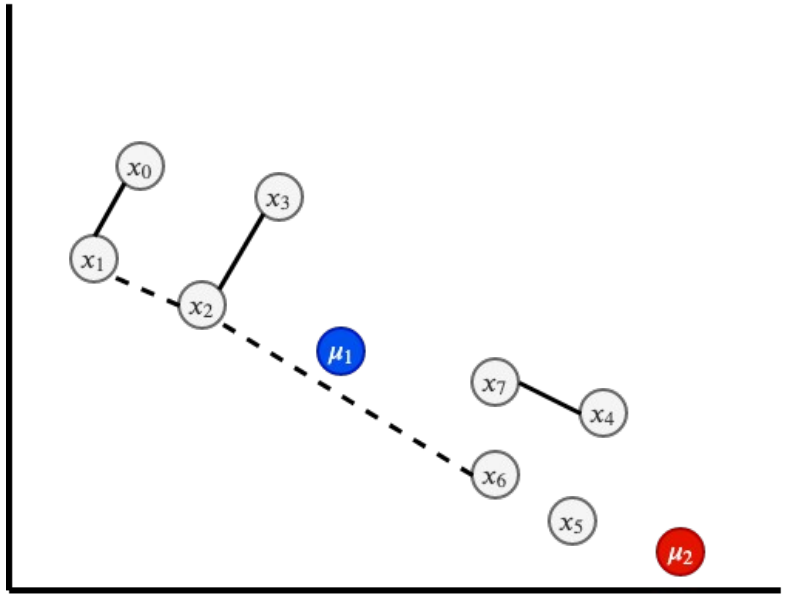
- El proceso itera hasta seleccionar los *m* elementos deseados

La Solución Greedy: Ejemplo

Supongamos que las instancias de X se exploran en el siguiente orden:

$$\overrightarrow{x_0} \rightarrow \overrightarrow{x_2} \rightarrow \overrightarrow{x_1} \rightarrow \overrightarrow{x_3} \rightarrow \overrightarrow{x_7} \rightarrow \overrightarrow{x_4} \rightarrow \overrightarrow{x_6} \rightarrow \overrightarrow{x_5}$$

Y supongamos $k = 2$ y la siguiente especificación de restricciones e inicialización de centroides:

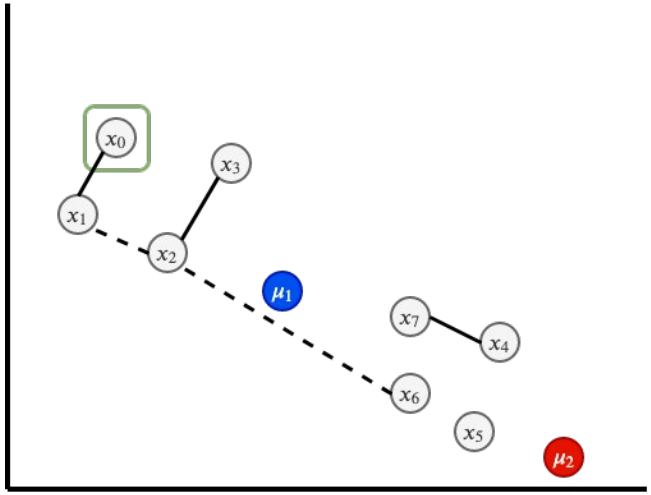


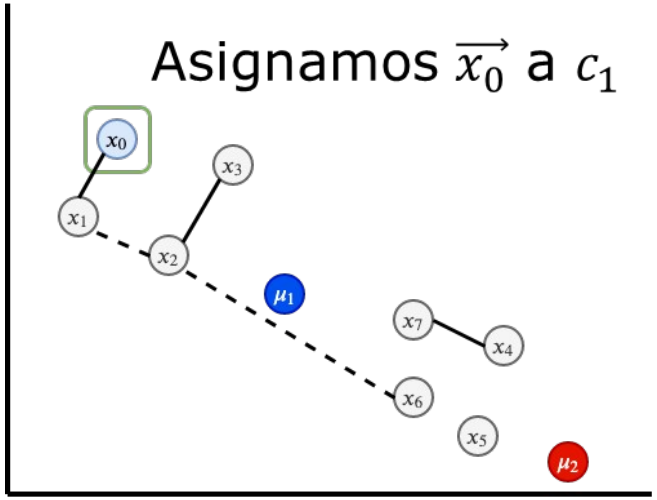
La Solución Greedy: Ejemplo

- $\overrightarrow{x_0} \rightarrow \overrightarrow{x_2} \rightarrow \overrightarrow{x_1} \rightarrow \overrightarrow{x_3} \rightarrow \overrightarrow{x_7} \rightarrow \overrightarrow{x_4} \rightarrow \overrightarrow{x_6} \rightarrow \overrightarrow{x_5}$

Asignar $\overrightarrow{x_0}$ a c_1 ⟶ $infeasibility = 0$

Asignar $\overrightarrow{x_0}$ a c_2 ⟶ $infeasibility = 0$





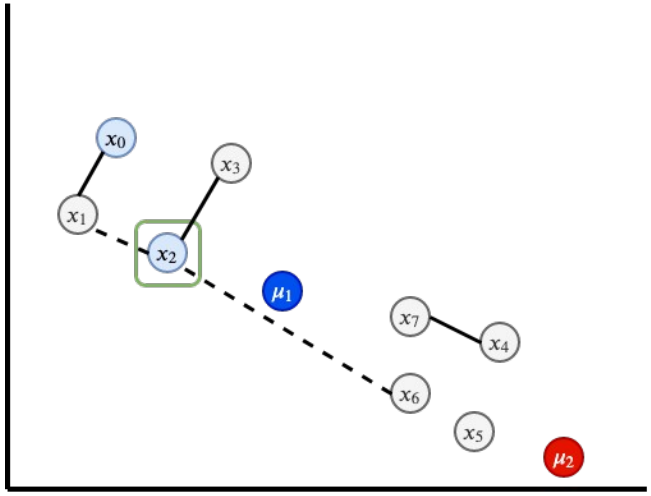
Ambos producen aumento de 0 en $inefeasibility$ así que asignamos $\overrightarrow{x_0}$ al cluster asociado con el centroide más cercano, c_1

La Solución Greedy: Ejemplo

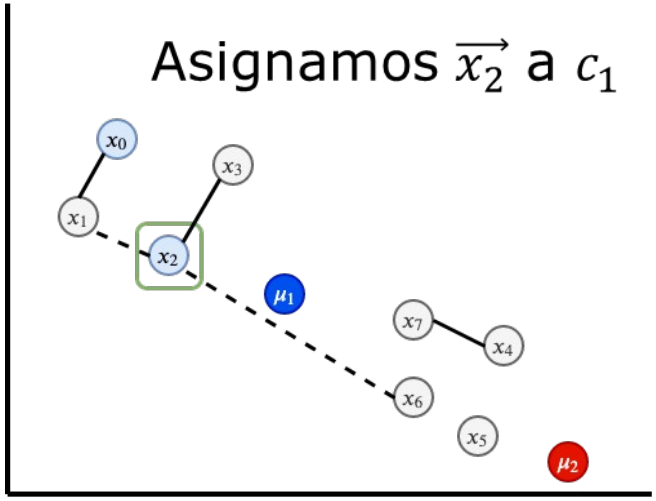
- $\overrightarrow{x_0} \rightarrow \overrightarrow{x_2} \rightarrow \overrightarrow{x_1} \rightarrow \overrightarrow{x_3} \rightarrow \overrightarrow{x_7} \rightarrow \overrightarrow{x_4} \rightarrow \overrightarrow{x_6} \rightarrow \overrightarrow{x_5}$

Asignar $\overrightarrow{x_2}$ a c_1 ⟶ $infeasibility = 0$

Asignar $\overrightarrow{x_2}$ a c_2 ⟶ $infeasibility = 0$



Asignamos $\overrightarrow{x_2}$ a c_1



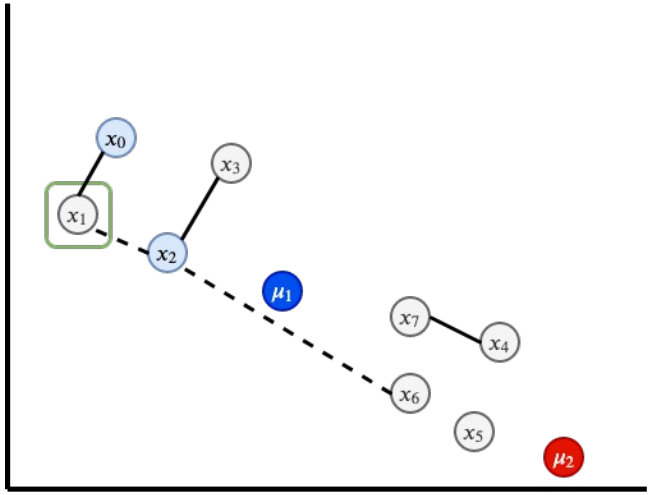
Ambos producen aumento de 0 en $inefeasibility$ así que asignamos $\overrightarrow{x_2}$ al cluster asociado con el centroide más cercano, c_1

La Solución Greedy: Ejemplo

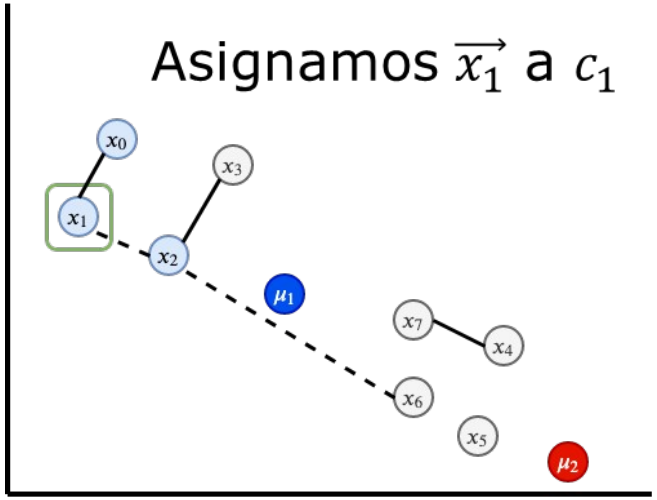
- $\overrightarrow{x_0} \rightarrow \overrightarrow{x_2} \rightarrow \overrightarrow{x_1} \rightarrow \overrightarrow{x_3} \rightarrow \overrightarrow{x_7} \rightarrow \overrightarrow{x_4} \rightarrow \overrightarrow{x_6} \rightarrow \overrightarrow{x_5}$

Asignar $\overrightarrow{x_1}$ a c_1 ⟶ $infeasibility = 1$ (Viola $CL(\overrightarrow{x_1}, \overrightarrow{x_2})$)

Asignar $\overrightarrow{x_1}$ a c_2 ⟶ $infeasibility = 1$ (Viola $ML(\overrightarrow{x_1}, \overrightarrow{x_0})$)



Asignamos $\overrightarrow{x_1}$ a c_1



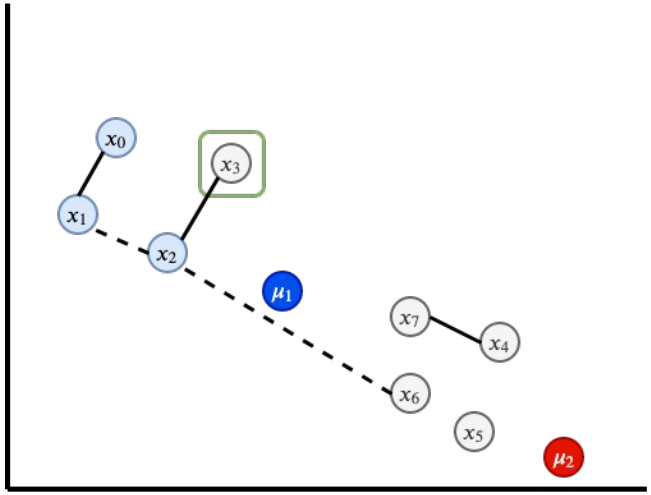
Ambos producen aumento de 1 en $inefeasibility$ así que asignamos $\overrightarrow{x_1}$ al cluster asociado con el centroide más cercano, c_1

La Solución Greedy: Ejemplo

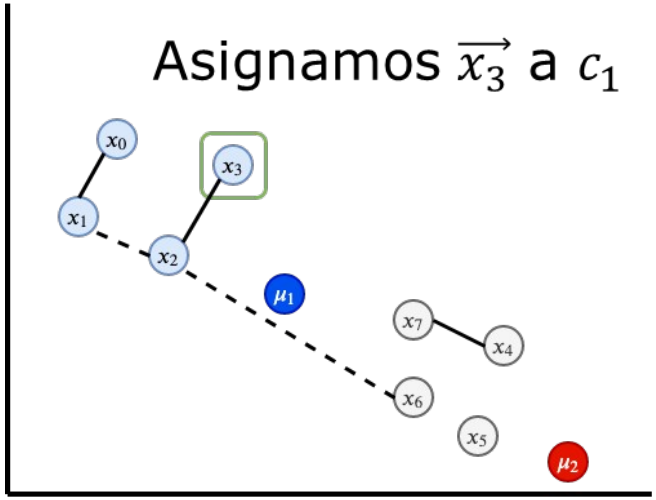
- $\overrightarrow{x_0} \rightarrow \overrightarrow{x_2} \rightarrow \overrightarrow{x_1} \rightarrow \overrightarrow{x_3} \rightarrow \overrightarrow{x_7} \rightarrow \overrightarrow{x_4} \rightarrow \overrightarrow{x_6} \rightarrow \overrightarrow{x_5}$

Asignar $\overrightarrow{x_3}$ a c_1 ⟶ $infeasibility = 1$

Asignar $\overrightarrow{x_3}$ a c_2 ⟶ $infeasibility = 2$ (Viola $ML(\overrightarrow{x_2}, \overrightarrow{x_3})$)



Asignamos $\overrightarrow{x_3}$ a c_1



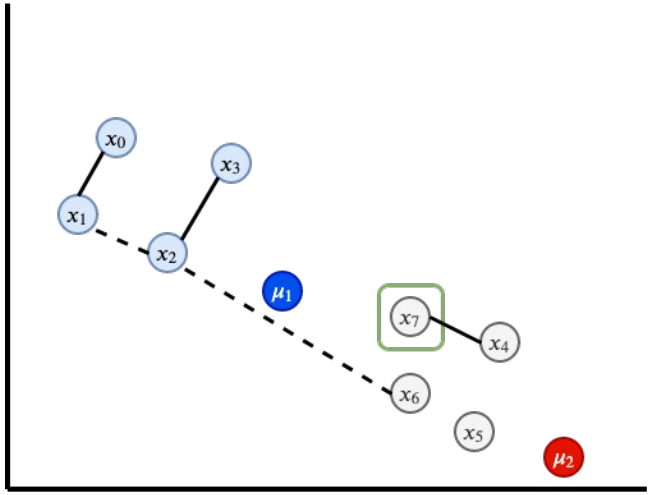
En este caso una de las asignaciones produce peor $infeasibility$ que otra. **Escogeremos siempre la que menos aumente $infeasbility$**, que en esta ocasión resulta ser la del cluster c_1, con el centroide más cercano

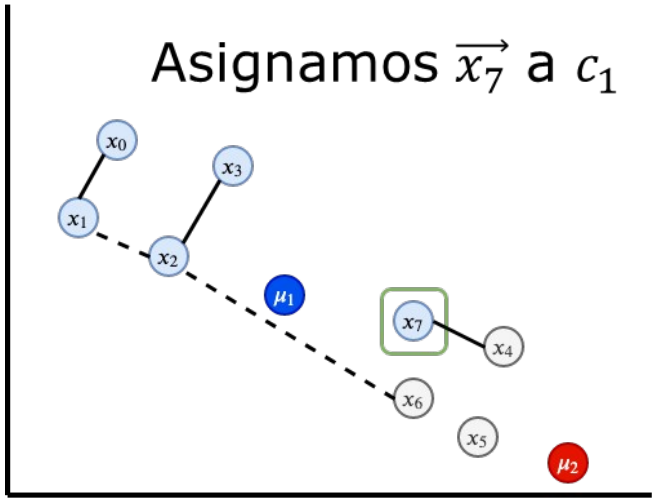
La Solución Greedy: Ejemplo

- $\overrightarrow{x_0} \rightarrow \overrightarrow{x_2} \rightarrow \overrightarrow{x_1} \rightarrow \overrightarrow{x_3} \rightarrow \overrightarrow{x_7} \rightarrow \overrightarrow{x_4} \rightarrow \overrightarrow{x_6} \rightarrow \overrightarrow{x_5}$

Asignar $\overrightarrow{x_7}$ a c_1 ⟶ $infeasibility = 1$

Asignar $\overrightarrow{x_7}$ a c_2 ⟶ $infeasibility = 1$





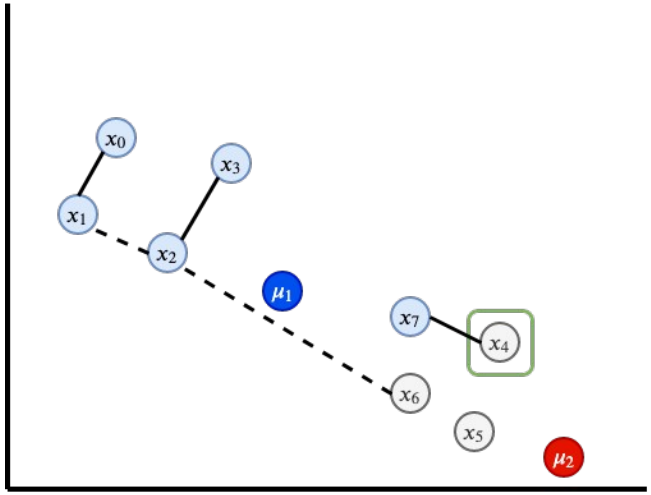
Ambos producen aumento de 0 en $inefeasibility$ así que asignamos $\overrightarrow{x_7}$ al cluster asociado con el centroide más cercano, c_1

La Solución Greedy: Ejemplo

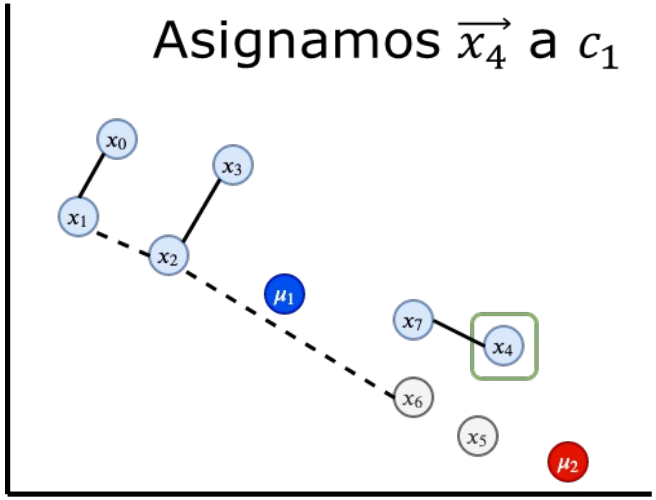
- $\overrightarrow{x_0} \rightarrow \overrightarrow{x_2} \rightarrow \overrightarrow{x_1} \rightarrow \overrightarrow{x_3} \rightarrow \overrightarrow{x_7} \rightarrow \overrightarrow{x_4} \rightarrow \overrightarrow{x_6} \rightarrow \overrightarrow{x_5}$

Asignar $\overrightarrow{x_4}$ a c_1 ⟶ $infeasibility = 1$

Asignar $\overrightarrow{x_4}$ a c_2 ⟶ $infeasibility = 2$ (Viola $ML(\overrightarrow{x_4}, \overrightarrow{x_7})$)



Asignamos $\overrightarrow{x_4}$ a c_1



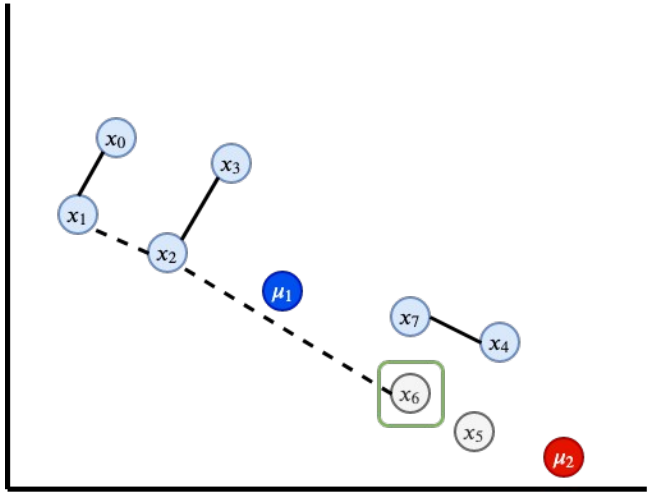
En este caso una de las asignaciones produce peor $infeasibility$ que otra. **Escogeremos siempre la que menos aumente $infeasbility$**, que en esta ocasión resulta ser la del cluster con el **segundo centroide más cercano**, c_1

La Solución Greedy: Ejemplo

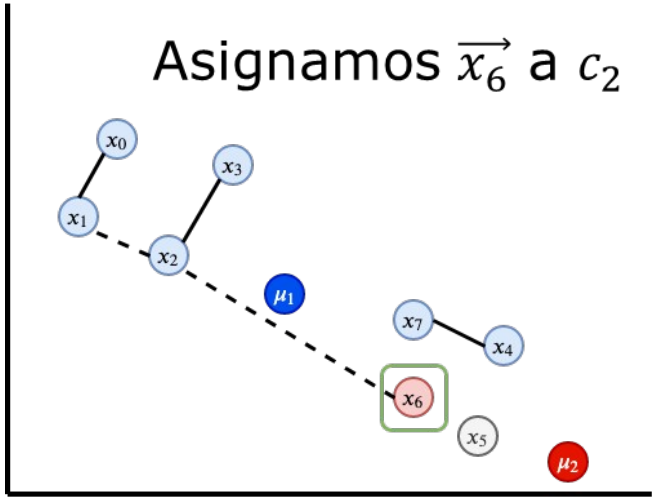
- $\overrightarrow{x_0} \rightarrow \overrightarrow{x_2} \rightarrow \overrightarrow{x_1} \rightarrow \overrightarrow{x_3} \rightarrow \overrightarrow{x_7} \rightarrow \overrightarrow{x_4} \rightarrow \overrightarrow{x_6} \rightarrow \overrightarrow{x_5}$

Asignar $\overrightarrow{x_6}$ a c_1 ⟶ $infeasibility = 2$ (Viola $CL(\overrightarrow{x_2}, \overrightarrow{x_6})$)

Asignar $\overrightarrow{x_6}$ a c_2 ⟶ $infeasibility = 1$



Asignamos $\overrightarrow{x_6}$ a c_2



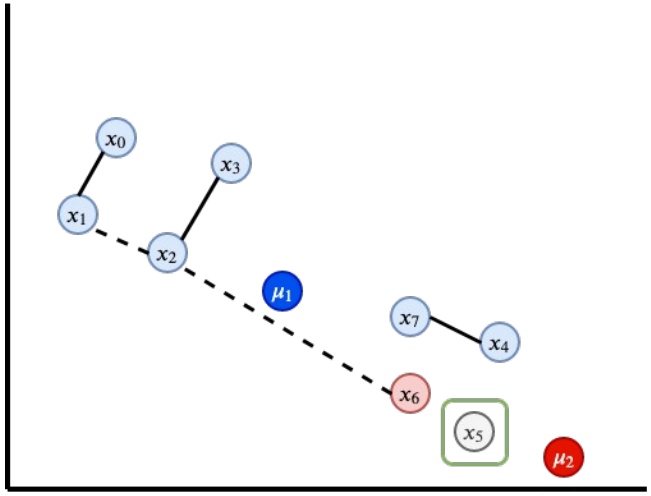
En este caso una de las asignaciones produce peor $infeasibility$ que otra. **Escogeremos siempre la que menos aumente $infeasbility$**, que en esta ocasión resulta ser la del cluster con el centroide más cercano, c_2

La Solución Greedy: Ejemplo

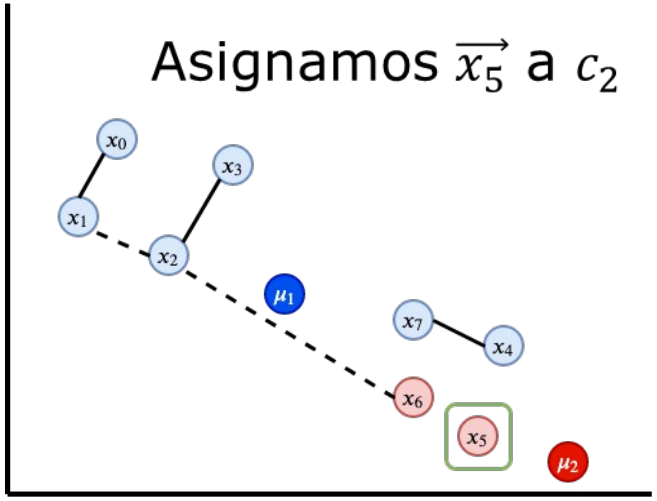
- $\overrightarrow{x_0} \rightarrow \overrightarrow{x_2} \rightarrow \overrightarrow{x_1} \rightarrow \overrightarrow{x_3} \rightarrow \overrightarrow{x_7} \rightarrow \overrightarrow{x_4} \rightarrow \overrightarrow{x_6} \rightarrow \overrightarrow{x_5}$

Asignar $\overrightarrow{x_5}$ a c_1 ⟶ $infeasibility = 1$

Asignar $\overrightarrow{x_5}$ a c_2 ⟶ $infeasibility = 1$



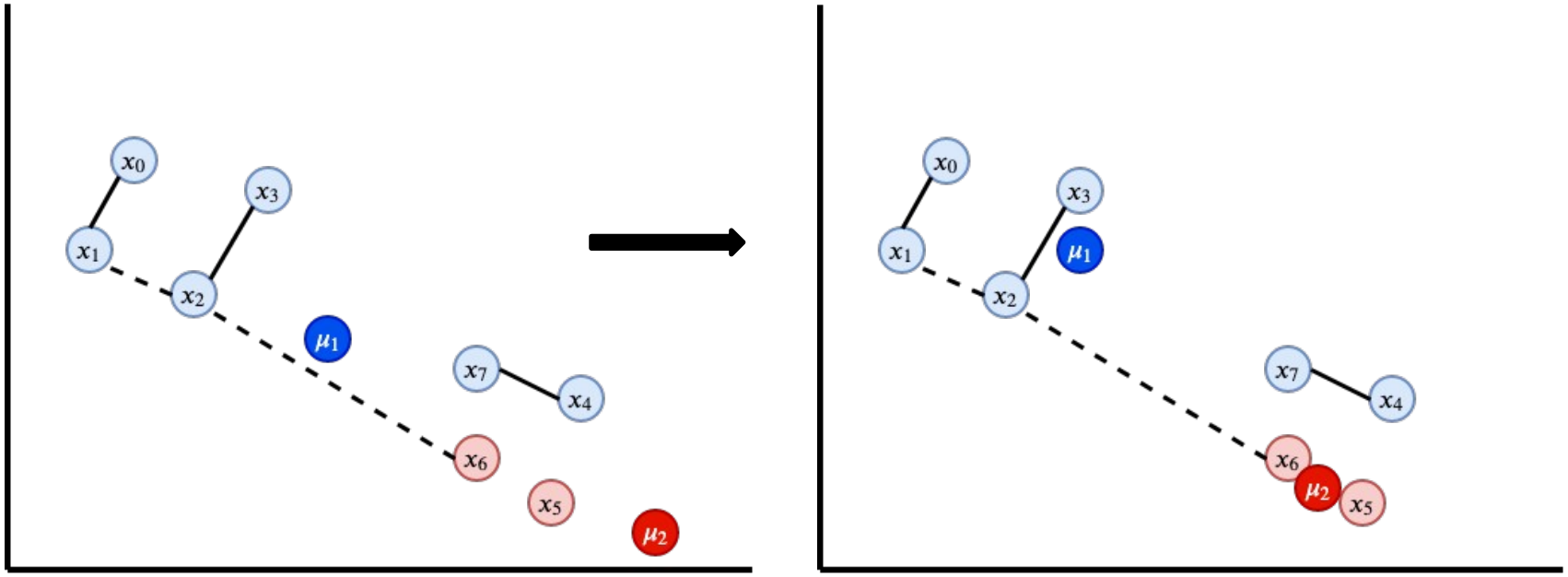
Asignamos $\overrightarrow{x_5}$ a c_2



Ambos producen aumento de 0 en $inefeasibility$ así que asignamos $\overrightarrow{x_5}$ al cluster asociado con el centroide más cercano, c_2

La Solución Greedy: Ejemplo

Ya se han explorado todas las instancias. Hay que actualizar los centroides:



Como se ha producido cambio en la partición el proceso de agrupamiento continúa

La Solución Greedy: Resumen

Para cada instancia $\overrightarrow{x_i}$, calculamos el cambio que se produce en $infeasibility$ cuando la asignamos a cada cluster c_i. De entre las asignaciones que menos aumento producen en $infeasibility$ elegimos la asociada al cluster c_j con el centroide más cercano $\overrightarrow{\mu_j}$

Este método es altamente dependiente del orden en el que se exploran las instancias de X. **Ordenes de exploración distintos producen particiones con distintos valores para $\boldsymbol{infeasibility}$ y para $\overline{C}$**

Búsqueda por Trayectorias Simples

El método de búsqueda por trayectorias simples que usaremos para aplicar al PAR es la **Búsqueda Local**

En este caso haremos una **interpretación débil de las restricciones**

Para construir un esquema de optimización por búsqueda local tenemos que definir la **función objetivo y el operador de vecino**, así como el esquema de representación del problema

Búsqueda por Trayectorias Simples: El Algoritmo

El algoritmo propuesto utiliza una estrategia tipo "**el primer mejor**" para explorar el entorno de una solución. Esto es, se acepta el primer vecino que mejore a la solución actual como nueva solución

La exploración del entorno no se hace en un orden determinístico sino en un orden aleatorio para dar diversidad

Para evaluar la calidad de las soluciones utilizamos la función objetivo *f*, que hay que minimizar. De esta forma se acepta como nueva solución el primer vecino S' que cumpla $f(S') < f(S)$

El proceso termina cuando se alcanza el número máximo de evaluaciones de la función objetivo o cuando no se produce mejoría al explorar un vecindario concreto ($\nexists\, S' t.q. f(S') < f(S)$)

La solución inicial se genera de manera aleatoria, comprobando que cada cluster tiene al menos una instancia

Búsqueda por Trayectorias Simples: Operador de Vecino

El operador de vecino considerado es el **cambio de asignación de cluster de una instancia ($\boldsymbol{Cambio_Cluster}(S, i, l)$)**. Dada una solución (configuración de clusters) S se escoge un índice i (instancia $\overrightarrow{x_i}$) y se cambia su asignación a otro cluster distinto l:

$$S = \{s_0, \ldots, \boldsymbol{s_i}, \ldots, s_n\} \quad \Rightarrow \quad S' = \{s_1, \ldots, l, \ldots, s_n\}$$

<u>Es necesario chequear que el cambio no deja ningún cluster vacío, ya que no verificaría las restricciones del problema del clustering</u>

El entorno de una solución S está formado por las soluciones accesibles desde ella a través de la aplicación de dicho operador sobre todos los índices de S, $i \in \{1, \ldots, n\}$, estudiando todos los posibles valores alternativos de s_i de X de un cluster a otro

Por tanto, **el tamaño del entorno de una solución es $\boldsymbol{n \cdot (k-1)}$**

Búsqueda por Trayectorias Simples: Operador de Vecino

Por ejemplo, para $n = 3$ *y* $k = 3$ *(tres clusters:* $l_i \in \{1,2,3\}\ \forall\, i \in \{1,\ldots,n\}$*), y dada una solución inicial* S*, el operador de vecino genera:*

$$S = [1, 2, 3]$$

Vecindario

$$S = [2, 2, 3]$$
$$S = [3, 2, 3]$$

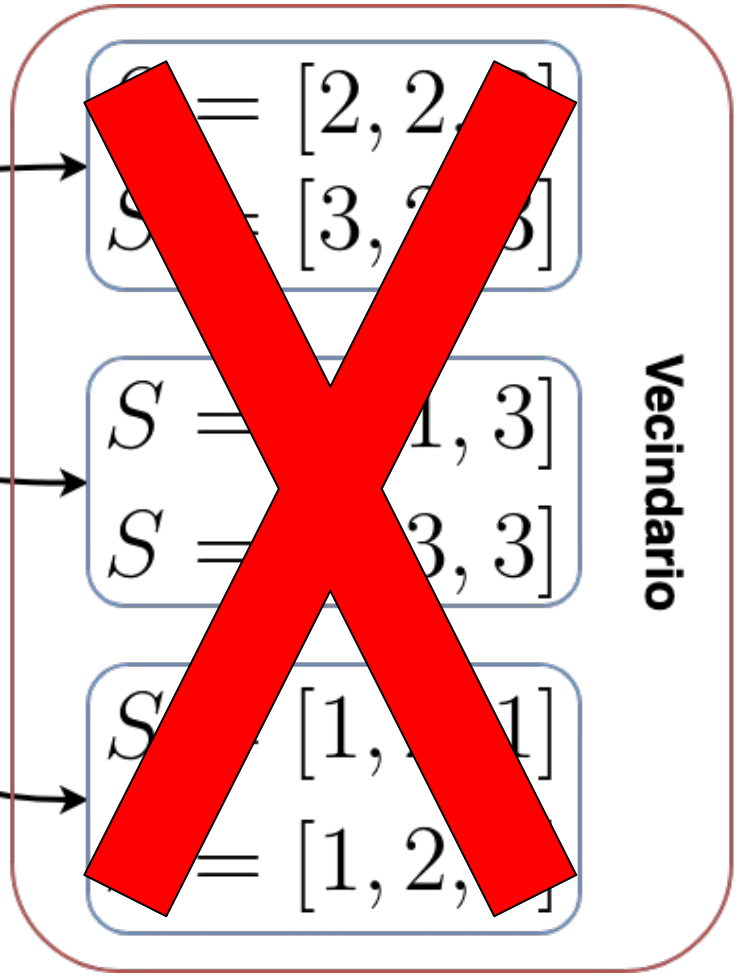
$$S = [1, 1, 3]$$
$$S = [1, 3, 3]$$

$$S = [1, 2, 1]$$
$$S = [1, 2, 2]$$

Búsqueda por Trayectorias Simples: Operador de Vecino

Por ejemplo, para $n = 3$ *y* $k = 3$ *(tres clusters:* $l_i \in \{1,2,3\}\ \forall\, i \in \{1,\ldots,n\}$*), y dada una solución inicial* S*, el operador de vecino genera:*

$$S = [1, 2, 3]$$



¡OJO! ESTOS VECINOS NO SERÍAN VÁLIDOS. NO CUMPLEN LA RESTRICCIÓN DE MANTENER 3 CLUSTERS

Búsqueda por Trayectorias Simples: El Algoritmo

No es necesario generar el entorno completo para explorarlo. En su lugar generamos un vecindario virtual que consiste en parejas índice-valor

Esto es, generamos los cambios posibles sobre la solución actual, no las nuevas soluciones en sí. Al momento de evaluar un vecino de la solución actual se aplica el cambio correspondiente y se evalúa. Gracias a este mecanismo la memoria ocupada por el vecindario es mucho menor

Otra ventaja de este mecanismo es que el proceso de aleatorización es más simple. En lugar de barajar el vecindario real para determinar el orden de exploración, barajamos el vecindario virtual (parejas de tamaño invariable)

¡Cuidado! **Barajar el entorno no es lo mismo que generarlo de forma aleatoria**. Si se barajan los individuos no se repiten, si se selecciona aleatoriamente sí se podrían repetir

Búsqueda por Trayectorias Simples: Ejemplo vecindarios

Un ejemplo de vecindario virtual para $n = 3$ y $k = 3$ (tres clusters: $l_i \in \{1,2,3\}\ \forall\, i \in \{1, \dots, n\}$), y dada una solución inicial $S = [1,2,3]$, el operador de vecindario genera:

Vecindario Real

$$S = [2, 2, 3] \quad S = [1, 1, 3] \quad S = [1, 2, 1]$$
$$S = [3, 2, 3] \quad S = [1, 3, 3] \quad S = [1, 2, 2]$$

Vecindario Virtual

$$(1, 2) \quad (2, 1) \quad (3, 1)$$
$$(1, 3) \quad (2, 3) \quad (3, 2)$$

El formato de los individuos del vecindario virtual es:

$$(indice, nuevo\ valor) = (i, l_i')$$

Búsqueda por Trayectorias Simples: Ejemplo vecindarios

Un ejemplo de vecindario virtual para $n = 3$ y $k = 3$ (tres clusters: $l_i \in \{1,2,3\}\ \forall\, i \in \{1, \dots, n\}$), y dada una solución inicial $S = [1,2,3]$, el operador de vecindario genera:

Vecindario Real

$S = [1, 1, 3]$

$S = [3, 2,$ $[1, 2, 2]$

$(2, 1)$

$(1, 3)$ $(2, 3)$ $(3, 2)$

ESTOS VECINOS NO SON VÁLIDOS. SÓLO GUARDAMOS LOS FACTIBLES

El formato de los individuos del vecindario virtual es:

$$(indice, nuevo\ valor) = (i, l_i')$$

Algunos consejos

- Las restricciones se dan en forma de matriz pero es recomendable construir la lista de restricciones y tener ambas disponibles para usar la más eficiente en cada caso

- Estamos trabajando con números reales así que hay que tener cuidado con las comparaciones de igualdad

- Un manera de saber si la implementación es correcta es comprobar que la calidad de la solución escala con el tamaño del conjunto de restricciones y con la infactibilidad conseguida (en el caso medio). Si no es así es posible que haya algún fallo

Agradecimientos

- Para la preparación de las transparencias de presentación del PAR se han usado materiales del estudiante de doctorado Germán González Almagro